SkillNavora · Business AI Stack

COMPLETE GUIDE 12 OF 15

AI Agents

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

AI AGENTS · COMPLETE GUIDE

AI agents: build one that reads, decides and acts, with you in control

An AI agent is a model in a loop: it looks at a goal, chooses a tool, reads the result, and continues until the goal is done or it needs you. This guide builds a complete job-search agent that reads recruiter emails, drafts replies, tailors applications, and waits for your approval before anything is sent or submitted.

1. What makes it an agent

- **Goal.** A clear outcome: "reply to recruiters and prepare applications for product roles".
- **Tools.** Functions it may call: read mail, search a sheet, draft a message, update a row, request approval.
- **Loop.** Plan → act → observe → repeat, with a step limit.
- **Memory.** What it may keep between runs: a sheet of decisions, not hidden memory.
- **Stop rules.** When to finish, and when to ask a human.

2. Agent or workflow?

Use a workflow (n8n, Make, Zapier) when	Use an agent when
Steps are fixed and known in advance	The steps depend on what the input says
Predictable, easy to test	Judgement is needed (which email is a recruiter, which role fits)

The best design often combines both: a workflow collects the data, and an agent makes the judgement calls inside it.

3. The job-search agent: design

Tool	Purpose and limit
read_new_emails	Returns the subject, sender and snippet of unread mail. No attachments.
get_profile	Returns your approved profile text (skills, experience, availability). The only source of facts about you.
list_roles / set_role_status	Reads and updates the job sheet (status: new, drafted, approved, sent).
draft_reply	Writes a draft into Gmail. It cannot send.
request_approval	Posts a summary to you (WhatsApp or Telegram) and stops until you reply APPROVE or EDIT.
send_approved	Sends an email only when the item is approved in the sheet. Limited to 10 sends a day.

4. Worked example: reply to a recruiter and prepare a job application

1. **Set the goal and rules in the system prompt:** *"You help one candidate. Use only facts in the profile. Never promise salary, dates or availability. Never send anything yourself; draft and request approval. Stop after 10 steps."*
2. **Trigger:** an n8n Schedule every 30 minutes runs the agent with the new emails.
3. **The agent's loop, for one email from a recruiter:**

```
1. read_new_emails -> finds: from "priya@acme.example", subject "Product Analyst role?"
2. get_profile     -> skills, years, availability, salary range (kept private)
3. classify        -> recruiter_outreach (role: Product Analyst, company: Acme)
4. list_roles      -> finds the matching row (status: new)
5. draft_reply     -> "Thank you, Priya. Yes, I am interested. I have 4 years in product analytics..."
6. set_role_status -> drafted
7. request_approval -> sends you: "Reply to Priya at Acme? Reply APPROVE, EDIT or SKIP."
8. stop            -> waits for your answer (the loop ends; a later run resumes)
```

4. **Your answer:** you reply APPROVE. A second workflow sees the approval in the sheet and calls send_approved; the sheet status becomes sent.
5. **Applications:** for a role you want, the agent tailors a cover note from the profile and the job description you paste in, sets status drafted, and asks for approval. Submission uses the company's official application email or form. Automated submission on portals whose terms prohibit bots is not allowed; do not build it.

5. Minimal agent code (Python, illustrative)

```
def run_agent(goal, tools, client, max_steps=10):
    messages = [{"role": "user", "content": goal}]
    for _ in range(max_steps):
        reply = client.messages.create(model=MODEL, system=RULES, tools=tool_specs(tools),
                                       messages=messages, max_tokens=800)
        if reply.stop_reason != "tool_use":           # the model is done
            return reply
        for block in reply.content:
            if block.type == "tool_use":
                if block.name == "send_approved":     # hard rule in code, not only in the prompt
                    raise PermissionError("sending needs a human approval")
                result = tools[block.name](**block.input)
                messages.append({"role": "assistant", "content": reply.content})
                messages.append({"role": "user", "content": [
                    {"type": "tool_result", "tool_use_id": block.id, "content": str(result)}]})
    return "stopped: step limit reached"
```

The loop is the pattern; take SDK names and tool-specification format from the current documentation of your provider.

THE KEY SAFETY RULE

Put limits in code, not only in the prompt. The prompt says "never send"; the code refuses a send without an approval record. A prompt can be ignored or manipulated; a code check cannot.

6. Evaluate the agent

- Run it on 30 saved emails. Mark each draft: correct class, no invented facts, right tone.
- Track errors per tool: wrong row, wrong recipient.
- Look at every step in the logs when something goes wrong.

7. Checklist

- Goal, tools and stop rules written down.
- Sending blocked in code until approved; daily send cap.
- Facts only from the profile; salary and dates never promised.
- Logs of every step; 30-email evaluation passed before going live.